

1. Sorting: Comparator

Comparators are used to compare two objects. In this challenge, you'll create a comparator and use it to sort an array. The *Player* class is provided in the editor below. It has two fields:

1. `name` : a string.
2. `score` : an integer.

Given an array of *Player* objects, write a comparator that sorts them in order of decreasing score. If or more players have the same score, sort those players alphabetically ascending by name. To do this, you must create a *Checker* class that implements the *Comparator* interface, then write an *int compare(Player a, Player b)* method implementing the [Comparator.compare\(T o1, T o2\)](#) method. In short, when sorting in ascending order, a comparator function returns `-1` if `a < b`, `1` if `a > b`, and `0` if `a == b`.

Declare a *Checker* class that implements the *comparator* method as described. It should sort first descending by score, then ascending by name. The code stub reads the input, creates a list of *Player* objects, uses your method to sort the data, and prints it out properly.

Example

Sort the list as follows. Sort first descending by score, then ascending by name.

Input Format

The first line contains an integer, `n`, the number of players.

Each of the next `n` lines contains a player's name, a string, and an integer.

Constraints

- Two or more players can have the same name.
- Player names consist of lowercase English alphabetic letters.

Output Format

You are not responsible for printing any output to stdout. Locked stub code in *Solution* will instantiate a *Checker* object, use it to sort the *Player* array, and print each sorted element.

Sample Input

```
5
```

```
amy 100
```

david 100

heraldo 50

aakansha 75

aleksa 150

Sample Output

aleksa 150

amy 100

david 100

aakansha 75

heraldo 50

Explanation

The players are first sorted descending by score, then ascending by name.

PROGRAM

```
import java.util.*;
```

```
class Player {
```

```
    String name;
```

```
    int score;
```

```
    Player(String name, int score) {
```

```
        this.name = name;
```

```
        this.score = score;
```

```
    }
```

```
}
```

```
class Checker implements Comparator<Player> {
```

```
    @Override
```

```
    public int compare(Player a, Player b) {
```

```
        // Score: decreasing order
```

```
        if (a.score != b.score) {
```

```
            return b.score - a.score;
```

```
        }
```

```
        // Name: alphabetical ascending order
```

```
        return a.name.compareTo(b.name);
```

```
    }
```

```
}
```

```
public class Solution {
```

```
    public static void main(String[] args) {
```

```
        Scanner scan = new Scanner(System.in);
```

```
        int n = scan.nextInt();
```

```
        Player[] player = new Player[n];
```

```
        Checker checker = new Checker();
```

```
        for(int i = 0; i < n; i++){
```

```
            player[i] = new Player(scan.next(), scan.nextInt());
```

```
        }
```

```
        scan.close();
```

```
        Arrays.sort(player, checker);
```

```

for(int i = 0; i < player.length; i++){

    System.out.printf("%s %s\n", player[i].name, player[i].score);

}

}

}

```

OUTPUT

The screenshot shows a web browser window with the URL <https://www.hackerrank.com/challenges/ctd-comparator-sorting/problem>. The page displays a green 'Congratulations' banner with the text 'You solved this challenge. Would you like to challenge your friends?' and a 'Next Challenge' button. Below the banner, a list of test cases is shown, all marked as 'Success'. The 'Test case 0' section is expanded, showing the 'Compiler Message' as 'Success'. The 'Input (stdin)' section displays the following data:

Line	Input
1	5
2	amy 100
3	david 100
4	heraldo 50
5	aakansha 75
6	aleksa 150

The 'Expected Output' section is also visible, with a 'Download' button. The browser's taskbar at the bottom shows the system time as 18:24 on 12-09-2026, with a temperature of 32°C and a battery level of 40%.